PROPOSTA DE TRABALHO · MODELAGEM E PROJETO DE ALGORITMOS PARA O MERCADO FINANCEIRO

Cata-vento Seletivo

Robô de day trade no Mini Índice, escrito em MQL5 para rodar no MetaTrader 5, que usa Random Forest para escolher quais sinais operar, com retreino em janela móvel feito dentro do próprio robô

4.1 Título e identificação

Título	**Cata-vento Seletivo**: robô de day trade no Índice Futuro (WIN) para MetaTrader 5, com filtro de sinais por Random Forest retreinado em janela móvel (*walk-forward*)
Integrantes	Luisa Caetano Araujo · RA 0076937 Raíssa Faria Ferreira · RA 0070311
Disciplina	Modelagem e Projeto de Algoritmos para o Mercado Financeiro
Professor	Dr. Alexandre Pimenta
Data de entrega	03/11/2026
Etapa	Proposta conceitual e metodológica. A implementação em MQL5 é a próxima etapa
Plataforma	MetaTrader 5: Expert Advisor em MQL5, Random Forest da biblioteca ALGLIB que acompanha o MT5, backtest no Testador de Estratégias

4.2 Objetivo e justificativa

Objetivo específico do robô

Operar day trade no **Mini Índice Futuro da B3** (série contínua `WIN$` no backtest), no tempo gráfico de **15 minutos**. Uma regra de seguimento de tendência baseada no **DMI/ADX** gera os *candidatos* a entrada. Um classificador **Random Forest** estima, para cada candidato, a probabilidade de a operação atingir o alvo antes do stop, e o robô só entra quando essa probabilidade passa de um limiar p^*. O modelo é **retreinado a cada 20 pregões** (cerca de um mês) com os 120 pregões imediatamente anteriores.

Todo o sistema é um único **Expert Advisor em MQL5**: a regra, as variáveis, a rotulagem, o treino do Random Forest, a escolha do limiar e a execução das ordens rodam dentro do MetaTrader 5. Não há etapa fora da plataforma. Assim, o mesmo arquivo que é testado no Testador de Estratégias é o que vai operar na conta.

Por que usar Aprendizado de Máquina

- **A mesma regra acerta em um contexto e erra em outro.** Um cruzamento de DI com ADX forte às 10h, logo após um gap, não é o mesmo sinal que o cruzamento às 16h num dia sem volatilidade. Uma

regra fixa trata os dois igual. O modelo aprende, a partir do histórico, em quais contextos o sinal funciona.
- **As interações são não lineares.** "ADX alto só ajuda quando a volatilidade é média" é o tipo de condição que ninguém escreve à mão, mas que uma árvore de decisão encontra sozinha.
- **O mercado muda de regime.** O que separa sinal bom de sinal ruim em 2024 pode não valer em 2026. Retreinar em janela móvel mantém o modelo atualizado.

Vantagem competitiva esperada

- **Menos operações ruins.** A regra base gera sinais em excesso de propósito. O filtro descarta os de baixa probabilidade, o que deve subir o fator de lucro e reduzir o rebaixamento, mesmo operando menos.
- **Base conhecida, com vantagem a provar.** A regra de entrada vem do robô *ADX Cata-vento* (Trabalho 01), que fez fator de lucro de 1,36 em 533 operações no WINV26 (77 pregões, dentro da amostra e sem custos) e 1,11 em 1.742 operações no `WIN$` em 2026, também sem custos. Com o custo desta proposta (R$ 3 por operação), esses fatores seriam 1,13 e 0,92. Uma medição preliminar e simplificada da regra desta proposta, sem o filtro e em barras de 1 minuto, deu 1,09 sem custos e 1,03 com custos de 2023 a 2026. A margem é fina, e é por isso que o filtro precisa operar menos e melhor: essa é a hipótese a testar.
- **Validação honesta.** No Trabalho 01, o resultado principal foi medido no próprio WINV26, sem período fora da amostra planejado, porque o contrato vence e o histórico some. O teste posterior no `WIN$` em 2026 ajuda, mas não é totalmente inédito. A série contínua permite dezenas de meses realmente fora da amostra.
- **Decisão explicável.** A importância das variáveis dá uma indicação do que o modelo está usando. Se ele passar a depender de algo sem sentido econômico, isso aparece.

Oportunidade de mercado

O Mini Índice é um dos contratos mais negociados da B3. Tem liquidez alta durante todo o pregão, margem baixa e valor de **R$ 0,20 por ponto** (variação mínima de 5 pontos), o que permite operar com capital pequeno e dimensionar posição em passos finos. O índice brasileiro tem movimentos intradiários direcionais frequentes, que são o terreno de uma estratégia de tendência. A obrigação de zerar no fim do dia elimina o risco de gap noturno.

4.3 Descrição da estratégia

Fundamentos

A estratégia segue tendência e tem duas camadas. A **camada de regra** usa o sistema DMI de Wilder: **DI+ e DI−** dizem *para onde* (qual lado manda) e o **ADX** diz se há força. O **ATR** mede a volatilidade corrente e define o tamanho do stop e do alvo. A **camada de aprendizado** recebe cada sinal da regra e decide se ele vale a pena naquele contexto.

Indicadores técnicos

INDICADOR	USO NA REGRA	USO COMO VARIÁVEL DO MODELO
DI+ / DI− (12)	Lado dominante define compra ou venda	Diferença DI+ − DI−, no sentido da operação
ADX (12)	Filtro mínimo de força (ADX ≥ 20)	Nível e inclinação do ADX
ATR (14)	Escala stop, alvo e trava no zero a zero	ATR relativo ao preço e seu percentil recente
VWAP do dia	—	Distância do preço à VWAP, em ATRs

DMI 12 e ADX ≥ 20 ficam fixos. No Trabalho 01 (WINV26, sem custos), o fator de lucro ficou entre 1,24 e 1,42 com o período do DMI variando de 8 a 20, então esse valor não é crítico. O limiar 20 é mais permissivo que o clássico 25 de propósito: a regra gera mais candidatos e a floresta faz a seleção fina.

Regras de entrada

Avaliadas apenas no fechamento de cada barra de 15 minutos, sobre barras já fechadas:

- **Candidato de compra:** DI+ > DI− e ADX ≥ 20 e ADX subindo em relação à barra anterior.
- **Candidato de venda:** DI− > DI+ e ADX ≥ 20 e ADX subindo em relação à barra anterior.
- **Filtro do modelo:** o robô entra somente se P(alvo antes do stop) ≥ p^*, onde P vem do Random Forest e p^* é escolhido na validação (seção 4.4).
- **Filtros operacionais:** horário entre 09:15 e 17:00, nenhuma posição aberta, limites diários não atingidos.

Regras de saída, com justificativa

SAÍDA	REGRA	JUSTIFICATIVA
Stop loss	1,5 × ATR a partir da entrada	Limita a perda a um múltiplo do ruído normal do mercado. Um stop fixo em pontos fica curto em dia volátil e largo em dia calmo.
Take profit	3,0 × ATR a partir da entrada	Relação 2:1 entre alvo e stop, próxima da usada no Trabalho 01 (375 / 150 pontos). Com ela, basta acertar mais de 1/3 das operações para cobrir as perdas antes dos custos.
Trava no zero a zero	Stop vai ao preço de entrada após 1,5 × ATR a favor	Impede que uma operação que já andou a favor vire perda. No Trabalho 01 a trava melhorou o modo de entrada por estado (1,26 → 1,34, no WINV26 e sem custos).
Fim do pregão	Zera tudo às 17:30	É day trade: não carrega posição para o dia seguinte, o que elimina o risco de gap noturno e a margem de custódia.

As mesmas regras de saída, inclusive a trava no zero a zero, definem o **rótulo** com que o modelo é treinado (seção 4.4). Assim o modelo aprende o mesmo evento que decide o resultado da operação real.

Gestão de risco e tamanho de posição

- **Toda ordem sai com stop e alvo** registrados no servidor da corretora no momento da entrada.
- **Tamanho por risco, modulado pela confiança do modelo:** o risco por operação é R = 0,25% do capital se $p^* \le P < 0{,}65$, e R = 0,5% se $P \ge 0{,}65$. Se o limiar escolhido no ciclo for $p^* \ge 0{,}65$, todas as entradas usam 0,5%. Ajustar o tamanho pela probabilidade estimada segue a ideia de *bet sizing* de López de Prado (2018, cap. 10).

```
contratos = max(1, min(5, ⌊ R × Capital / (1,5 × ATR × R$ 0,20) ⌋ ))
```

Com capital de R$ 20.000, R = 0,5% e ATR de 380 pontos (mediana medida nos candidatos do `WIN$`, 2023–2026), o stop é de 570 pontos, R$ 114 por contrato: ⌊100 / 114⌋ = 0, e vale o mínimo de 1 contrato, com risco real de 0,57% do capital. Nesse capital, a fórmula dá 1 contrato em praticamente todos os casos, e o risco em reais sobe com a volatilidade. O ajuste pela confiança do modelo só passa a mudar o tamanho a partir de cerca de R$ 46 mil, quando o risco tende a ficar constante.

- **Trava diária:** no máximo 4 operações por dia. O robô para no dia após perder 1,5% do capital ou após 3 stops seguidos.
- **Uma posição por vez**, sem preço médio e sem aumentar posição perdedora.

4.4 Sistema de aprendizado de máquina

Técnica escolhida: Random Forest

O problema é de **classificação**: dado o contexto do mercado no momento de um sinal, a operação vai atingir o alvo antes do stop? O Random Forest (Breiman, 2001) é um conjunto de árvores de decisão, cada uma treinada numa amostra diferente dos dados e com um subconjunto aleatório das variáveis. A probabilidade final é a fração de árvores que votam em y = 1: na ALGLIB, cada árvore cresce até a folha ficar pura, e a folha guarda só a classe. Como o treino é balanceado, P serve para ordenar os sinais e não é a chance real de bater o alvo; a curva de calibração acompanha essa diferença.

A arquitetura é a que López de Prado (2018, cap. 3) chama de **meta-rotulagem** (*meta-labeling*): um modelo primário, aqui a regra do DMI/ADX, decide o *lado* da operação, e um modelo secundário, o Random Forest, decide *se* entra e com que tamanho. O modelo de ML não precisa prever a direção do mercado, só se aquele sinal específico costuma dar certo naquele contexto, o que é um problema bem mais fácil.

Escolhemos a técnica por cinco motivos:

- **Funciona com poucos dados.** Seis meses geram cerca de 1.700 candidatos, mas eles são muito parecidos entre si e só cerca de 12% batem o alvo, então a informação real é bem menor que esse número. O Random Forest lida bem com esse tamanho. Uma rede LSTM precisaria de muito mais exemplos e tenderia a decorar ruído.
- **Resiste ao sobreajuste.** A média de muitas árvores diferentes reduz a variância de uma árvore isolada (Breiman, 2001; Hastie, Tibshirani e Friedman, 2009, cap. 15).
- **Capta interações não lineares** entre variáveis sem que precisemos especificá-las, e dispensa normalização.
- **Já existe pronto dentro do MetaTrader 5.** A **ALGLIB**, biblioteca numérica distribuída com o MT5 em `MQL5\Include\Math\Alglib`, implementa o Random Forest na classe `CDForest`: treino (`DFBuilderBuildRandomForest`), previsão de probabilidade (`DFProcess`), importância das variáveis e gravação do modelo em arquivo (`DFSerialize`). Por isso o robô inteiro pode ser escrito em MQL5, sem Python e sem bibliotecas externas.
- **Treina em segundos.** Com alguns milhares de exemplos e 17 variáveis, cada floresta treinou em menos de 5 segundos em teste com a ALGLIB do MT5 (até 5.000 linhas), então retreinar a cada ciclo

tem custo baixo. Um Algoritmo Genético precisaria de um backtest por indivíduo, o que custaria horas por ciclo.

As alternativas foram descartadas por motivos específicos. **Redes neurais (LSTM)** exigem muitos dados, são lentas de treinar, sensíveis a hiperparâmetros e opacas, e em MQL5 teriam de ser escritas à mão ou treinadas fora da plataforma. O **Algoritmo Genético** otimiza parâmetros fixos, mas não olha o contexto de cada operação. A **Lógica Fuzzy** depende de funções de pertinência definidas à mão. Treinar fora (em Python) e carregar o modelo pronto no MT5 em formato ONNX também foi descartado: o modelo ficaria congelado, e o retreino a cada ciclo exigiria um processo fora da plataforma.

Variáveis de entrada (features)

Calculadas no fechamento da barra do sinal, só com informação disponível naquele instante. As variáveis de direção são multiplicadas pelo sentido da operação (+1 compra, −1 venda), para que "a favor" signifique o mesmo nos dois lados e um único modelo sirva para compra e venda. São 17 variáveis no total.

GRUPO	VARIÁVEL	COMO SAI NO MQL5
Tendência	ADX; inclinação do ADX em 3 barras; (DI+ − DI−) × sentido	buffers do mesmo indicador ADX do Trabalho 01 (`CopyBuffer`)
Volatilidade	ATR / preço; percentil do ATR nos últimos 20 pregões; amplitude do dia até agora em ATRs	`iATR` + barras M15 (`CopyRates`)
Momento	Retorno das últimas 1, 4 e 16 barras × sentido, em ATRs	barras M15
Posição no dia	Distância à VWAP × sentido, em ATRs; posição do preço na máxima e mínima do dia (0 a 1); gap de abertura × sentido, em ATRs	VWAP calculada no robô com o volume real (`real_volume`) das barras do dia
Fluxo	Volume da barra ÷ média do mesmo horário nos últimos 20 pregões	barras M15
Calendário	Minutos desde a abertura; dia da semana	`TimeToStruct`
Sinal	Sentido (+1/−1); candidatos já ocorridos no dia	contagem pela regra, sem depender do filtro

Rótulo (o que o modelo aprende)

Cada sinal da regra base no histórico é simulado com as mesmas saídas da seção 4.3 (stop, alvo, trava no zero a zero e encerramento às 17:30), numa adaptação do método das três barreiras (López de Prado, 2018, cap. 3):

```
y = 1  se a operação sai no alvo (3,0 × ATR)
y = 0  se sai no stop (1,5 × ATR), na trava no zero a zero ou no encerramento das 17:30
```

No histórico, **toda barra que satisfaz a regra vira um exemplo**, mesmo quando a barra anterior também era candidata. É o que acontece na operação real: se o modelo recusa um sinal, a barra seguinte é avaliada de novo. Exemplos consecutivos do mesmo movimento são parecidos entre si, por isso a

validação usa sempre um bloco de tempo posterior, nunca uma amostra aleatória. Já a simulação das operações (validação e execução) respeita uma posição por vez e o limite diário de 4 operações.

Como o robô calcula o rótulo: a simulação percorre as **barras de 1 minuto** do restante do pregão (`CopyRates` em `PERIOD_M1`), e não as de 15 minutos, para saber o que foi tocado primeiro. Se uma mesma barra de 1 minuto toca o stop e o alvo, conta como stop. É uma premissa conservadora: na dúvida, o modelo aprende o pior caso.

O que se pretende otimizar

- **A seleção das operações:** o modelo aprende a probabilidade $P(y = 1 \mid \text{contexto})$.
- **O limiar de entrada *p**, entre 0,40 e 0,70 em passos de 0,025 (13 valores): define quantos sinais passam pelo filtro.
- **Os hiperparâmetros do Random Forest**, nos termos que a ALGLIB oferece (tabela abaixo).

HIPERPARÂMETRO E FUNÇÃO NA ALGLIB	VALORES	JUSTIFICATIVA
Fração do treino sorteada por árvore `DFBuilderSetSubsampleRatio`	`0,20 · 0,35 · 0,50`	É o principal controle de sobreajuste da ALGLIB, que não limita a profundidade das árvores. A documentação da biblioteca indica reduzir a fração para reduzir o sobreajuste. López de Prado (2018, cap. 6) recomenda o mesmo quando os exemplos se sobrepõem no tempo, como os nossos.
Variáveis sorteadas por divisão `DFBuilderSetRndVars`	`4 (≈ √17) · 8`	$\sqrt{p}$ é o padrão para classificação (Hastie, Tibshirani e Friedman, 2009) e o padrão da própria ALGLIB. Junto com a fração da amostra, é o hiperparâmetro que mais muda o resultado (Probst, Wright e Boulesteix, 2019).
Número de árvores `DFBuilderBuildRandomForest`	`100, fixo`	Não precisa ser otimizado: em Brier e log loss, mais árvores não pioram; em taxa de erro e AUC pode haver pequenas oscilações, mas ajustar o número de árvores não é recomendado (Probst e Boulesteix, 2018). Acima de 64 a 128 árvores o ganho é desprezível (Oshiro, Perez e Baranauskas, 2012).
Semente aleatória `DFBuilderSetSeed`	`fixa`	O mesmo backtest dá sempre o mesmo resultado, o que permite conferir e comparar.

- **Desbalanceamento de classes:** uma medição preliminar no `WIN$` dá cerca de 12% de exemplos y = 1. A ALGLIB não aceita peso por classe, então o robô **replica as linhas da classe minoritária** (cerca de 7 vezes) só na matriz de treino, até as classes terem peso próximo, o que se aproxima do peso por classe no critério de Gini. A validação usa os dados sem réplicas.

São 6 configurações × 13 limiares = **78 combinações por ciclo**, uma grade pequena de propósito. Quanto mais combinações se testam no mesmo trecho de dados, maior a chance de a melhor ser só sorte (Bailey *et al.*, 2014).

Assim, o aprendizado de máquina otimiza a cada ciclo os parâmetros que decidem *se* a estratégia entra (o limiar *p** e a configuração do modelo), atendendo ao requisito de usar ML para otimizar parâmetros.

As regras de DMI/ADX e as saídas em ATR ficam fixas. Isso mantém o rótulo estável e evita otimizar a regra e o filtro ao mesmo tempo, o que multiplicaria o risco de sobreajuste.

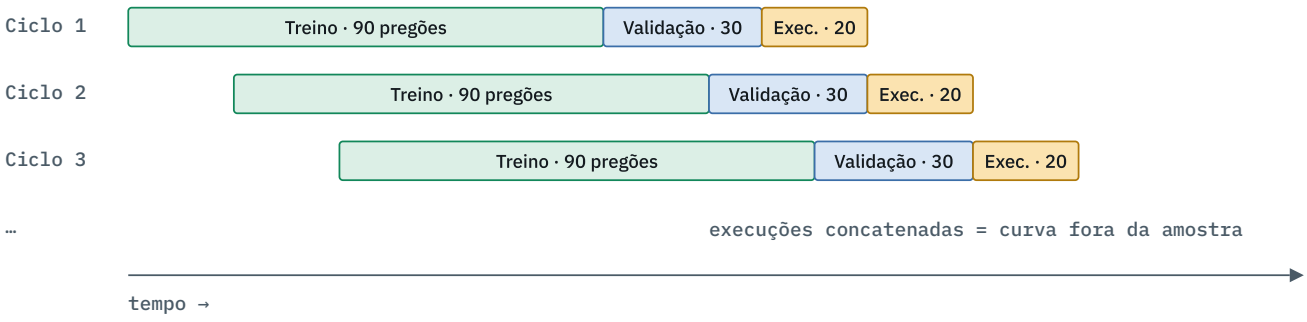
Função objetivo

São dois níveis. As **árvores** são construídas minimizando a impureza de Gini, o critério padrão de classificação e o que a ALGLIB usa. A **escolha de hiperparâmetros e limiar** não usa acurácia, porque acurácia alta não garante dinheiro. Ela usa o resultado financeiro das operações filtradas no trecho de validação, já com custos:

```
F(θ, p*) = LucroLíquido / RebaixamentoMáximo   (fator de recuperação)
sujeito a: N ≥ 20 operações e fator de lucro ≥ 1,05; se violar, F = −1
```

- **Fator de recuperação:** premia ganhar muito arriscando pouco. Maximizar só o lucro escolheria filtros que ganham com rebaixamentos insuportáveis. É a mesma métrica que o relatório do Testador de Estratégias do MT5 mostra, o que facilita conferir.
- **N ≥ 20:** impede que o limiar suba tanto que sobrem três operações de sorte.
- **Custos:** R$ 0,50 por contrato por lado de emolumentos e corretagem, mais 1 tick (5 pontos) de derrapagem por lado. É uma premissa conservadora.
- **Métricas de acompanhamento**, que não decidem nada: AUC, precisão e a curva de calibração das probabilidades, medidas na validação. O erro fora da bolsa (*out-of-bag*) da ALGLIB não é usado como evidência: as réplicas podem cair dos dois lados do sorteio e deixá-lo otimista. Em teste com dados aleatórios e fração 0,35, ele mostrou 7,5% de erro, quando o erro real era 50%. Pelo mesmo motivo, a importância das variáveis é só indicativa.

Arquitetura temporal: janela móvel (walk-forward)



Janela de treinamento de 120 pregões: os 90 primeiros ajustam o modelo e os 30 últimos escolhem hiperparâmetros e limiar. Depois o modelo é retreinado nos 120 pregões completos e opera nos 20 seguintes. Tudo avança 20 pregões e o ciclo se repete.

- **Janela de treinamento:** 120 pregões contínuos (cerca de 6 meses e 1.700 candidatos). Os 90 primeiros treinam candidatos, e os 30 últimos, sempre posteriores, escolhem hiperparâmetros e p^* pela função objetivo. O modelo final é retreinado nos 120 pregões com a configuração vencedora.
- **Janela de execução:** os 20 pregões seguintes (cerca de 1 mês). O robô opera com o modelo congelado.
- **Deslocamento:** ao fim de cada execução, as janelas avançam 20 pregões e o modelo é retreinado. O próprio robô faz isso, no primeiro tick do pregão, antes das 09:15 (seção 4.5).

- **Período de estudo:** `WIN$` M15 de janeiro de 2023 a outubro de 2026, cerca de 960 pregões e **~40 ciclos** fora da amostra.

Por que esses tamanhos de janela

ESCOLHA	JUSTIFICATIVA
Treino de **120 pregões**	É um meio-termo entre dois erros. Janela curta acompanha a mudança de regime, mas tem poucos exemplos e o modelo varia muito de um ciclo para o outro; janela longa é estável, mas mistura regimes que já não valem. Esse dilema é o problema central do aprendizado com *concept drift*, e janelas deslizantes de tamanho fixo são a solução clássica (Gama *et al.*, 2014). Seis meses dão cerca de 1.700 candidatos (medição preliminar; o menor valor de 2023 a 2026 foi 1.661). **Regra prévia:** se a contagem no histórico mostrar menos de 200 candidatos por janela, o treino passa para 180 pregões, decidido antes de ver qualquer resultado fora da amostra.
Validação de **30 pregões**, sempre depois do treino	Em série temporal, a validação tem de vir depois do treino: embaralhar os dados deixaria o modelo "ver o futuro" (Tashman, 2000; López de Prado, 2018, cap. 7). Trinta pregões é o tamanho que, com o filtro ligado, ainda deve render as 20 operações que a função objetivo exige: sem o filtro, a medição preliminar dá de 70 a 97 operações em 30 pregões.
Execução de **20 pregões**, avançando 20	É a análise walk-forward de Pardo (2008): otimiza num trecho, opera no trecho seguinte, que o modelo nunca viu, e repete. Como o deslocamento é igual à execução, as janelas de execução não se sobrepõem e, emendadas, formam uma única curva fora da amostra. Um mês por ciclo mantém o modelo atualizado e ainda dá cerca de 40 ciclos para avaliar.
Fronteira das janelas no **fim do pregão**	Todo rótulo se resolve até as 17:30 do mesmo dia, então nenhum exemplo do treino usa preços da validação ou da execução. É a condição de "purga" de López de Prado (2018, cap. 7), satisfeita aqui sem precisar descartar exemplos.
Robustez, não escolha	O backtest também será rodado com treino de 60 e de 250 pregões. Esses resultados serão **relatados, e não usados para escolher** a janela, porque escolher a melhor depois de ver o resultado seria sobreajuste à janela (Bailey *et al.*, 2014).

Restrições do problema

- **Operacionais:** apenas day trade, com tudo zerado às 17:30. Uma posição por vez. Toda ordem com stop e alvo. Máximo de 4 operações por dia.
- **Sem olhar o futuro:** as variáveis usam só barras fechadas. Validação sempre posterior ao treino, sem embaralhar a ordem temporal. A janela de execução nunca influencia o modelo. No Testador de Estratégias essa garantia é reforçada pela própria plataforma: o robô só enxerga os dados até o instante simulado.
- **Sem vazamento de rótulo:** toda operação simulada fecha até as 17:30 do mesmo pregão, então o rótulo de um sinal nunca usa dados de outro dia. Nenhum rótulo do treino depende de preços da validação ou da execução.
- **Estatísticas:** desbalanceamento de classes tratado pela replicação da classe minoritária. Mínimo de 20 operações na validação para aceitar uma configuração.
- **Da série contínua:** o `WIN$` emenda contratos com vencimentos diferentes. Por ser day trade, nenhuma posição atravessa a rolagem, e as variáveis que dependem de vários dias são recalculadas a partir do novo contrato.

4.5 Implementação no MetaTrader 5

> O robô é um **Expert Advisor (EA) em MQL5**. Tudo o que foi descrito acima, inclusive o treino do Random Forest e o laço walk-forward, roda dentro dele. O mesmo arquivo `.ex5` é usado no Testador de Estratégias e, depois, num gráfico de uma conta da B3, sem nenhuma etapa fora do MetaTrader 5.

- **Treino:** usa o Random Forest da ALGLIB, que acompanha o MT5, sem Python nem biblioteca externa. Em teste, cada floresta treinou em menos de 5 segundos, com até 5.000 linhas.
- **Backtest:** um único teste no Testador de Estratégias, com `WIN$` M15 de janeiro de 2023 a outubro de 2026, já é o walk-forward completo. Os primeiros 120 pregões são só aquecimento.
- **Conta:** os indicadores vêm do `WIN$`, e as ordens vão para o contrato vigente. Antes do dinheiro real, são pelo menos 20 pregões em conta demonstração.
- **Conferência:** com o filtro desligado e os parâmetros do Trabalho 01, o robô tem de reproduzir as operações no WINV26. Com a semente fixa, dois testes têm de dar o mesmo relatório.

O código e o plano de testes detalhados são a próxima etapa do trabalho.

---

5 Avaliação

Como o resultado será medido

Só a curva **fora da amostra** (a concatenação das janelas de execução) conta como resultado. Ela será comparada com duas referências no mesmo período e com os mesmos custos:

1. **Regra sem filtro:** o mesmo robô com o filtro desligado por um parâmetro de entrada, operando todos os sinais da regra base. Essa comparação isola o valor do Random Forest.
2. **Cata-vento original:** o robô do Trabalho 01 com seus parâmetros fixos.

Métricas: lucro líquido, fator de lucro, rebaixamento máximo, fator de recuperação, taxa de acerto, número de operações e AUC fora da amostra. A hipótese se confirma se o filtro superar a regra sem filtro em fator de recuperação na curva fora da amostra completa e na maioria dos ciclos; ciclo em que o filtro não opera conta como não superado. Se não superar, isso também é um resultado e será relatado como tal.

---

6 Referências

- BAILEY, David H.; BORWEIN, Jonathan M.; LÓPEZ DE PRADO, Marcos; ZHU, Qiji Jim. Pseudo-mathematics and financial charlatanism: the effects of backtest overfitting on out-of-sample performance. *Notices of the American Mathematical Society*, v. 61, n. 5, p. 458–471, 2014.
- BOCHKANOV, Sergey. *ALGLIB: decision forest (random forest)*. Manual de referência. Disponível em: alglib.net. Versão em MQL5 distribuída com o MetaTrader 5 em `MQL5\Include\Math\Alglib`.
- BREIMAN, Leo. Random Forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
- GAMA, João; ŽLIOBAITĖ, Indrė; BIFET, Albert; PECHENIZKIY, Mykola; BOUCHACHIA, Abdelhamid. A survey on concept drift adaptation. *ACM Computing Surveys*, v. 46, n. 4, art. 44, 2014.
- HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. *The Elements of Statistical Learning*. 2. ed. New York: Springer, 2009. Cap. 15: Random Forests.

- LÓPEZ DE PRADO, Marcos. *Advances in Financial Machine Learning*. Hoboken: Wiley, 2018. Caps. 3 (rotulagem e meta-rotulagem), 6 (métodos de conjunto), 7 (validação cruzada com purga) e 10 (tamanho da aposta).
- METAQUOTES. *MQL5 Reference*: Biblioteca Padrão (ALGLIB, CTrade) e Testador de Estratégias. Disponível em: mql5.com/pt/docs.
- OSHIRO, Thais M.; PEREZ, Pedro S.; BARANAUSKAS, José A. How many trees in a random forest? In: *Machine Learning and Data Mining in Pattern Recognition (MLDM 2012)*. Berlin: Springer, 2012. LNCS 7376, p. 154–168.
- PARDO, Robert. *The Evaluation and Optimization of Trading Strategies*. 2. ed. Hoboken: Wiley, 2008.
- PROBST, Philipp; BOULESTEIX, Anne-Laure. To tune or not to tune the number of trees in random forest. *Journal of Machine Learning Research*, v. 18, n. 181, p. 1–18, 2018.
- PROBST, Philipp; WRIGHT, Marvin N.; BOULESTEIX, Anne-Laure. Hyperparameters and tuning strategies for random forest. *WIREs Data Mining and Knowledge Discovery*, v. 9, n. 3, e1301, 2019.
- TASHMAN, Leonard J. Out-of-sample tests of forecasting accuracy: an analysis and review. *International Journal of Forecasting*, v. 16, n. 4, p. 437–450, 2000.
- WILDER, J. Welles. *New Concepts in Technical Trading Systems*. Greensboro: Trend Research, 1978.
- B3. Contrato Futuro Mini de Ibovespa: especificações. Disponível em: b3.com.br.